_____

(Nombre de la institución)

REPORTE TÉCNICO

Simulador de escape de incendios

Agente inteligente basado en metas para la búsqueda de rutas de evacuación con A y comparación con cinco estrategias de búsqueda*

Materia: _____

Tema: Estrategias de búsqueda ciega y heurística (proyecto #33)

Profesor(a): _____

Autores:

_____ (Parte 1. Mapas, cuadrícula y estructura)

_____ (Parte 2. BFS, DFS y UCS)

_____ (Parte 3. Greedy, A* y Hill Climbing)

_____ (Parte 4. Métricas, gráficas e interfaz)

Repositorio: github.com/luarmoro123/proyecto-ia

Fecha: ____ de _____ de 2026

Índice general

1. RESUMEN 6
1.1. Síntesis del proyecto 6
2. INTRODUCCIÓN 7
2.1. Contexto teórico 7
2.2. Planteamiento del problema 7
2.3. Diagrama de caso de uso 8
3. DESARROLLO 10
3.1. Arquitectura del agente inteligente 10
3.1.1. Identificación y clasificación del tipo de agente 10
3.1.2. Matriz de diagnóstico del modelo PEAS 10
3.2. Propiedades del entorno de búsqueda 11
3.4. Diagrama de componentes del caso a desarrollar 11
3.5. Abstracción del espacio de estados 12
4. ALGORITMO DE BÚSQUEDA UTILIZADO 15
4.1. Descripción de la estrategia seleccionada 15
4.2. Formulación de costos 15
5. CÓDIGO FUENTE 17
5.1. Implementación de clases y funciones 17
a) Sensores del agente: clase Cuadricula 17
b) Actuadores: conjunto de movimientos 17
c) Expansión de nodos: algoritmo A* 18
d) Rastreo inverso (backtracking) del camino óptimo 18
e) Componente de aprendizaje: memoización 19
6. CAPTURA DE PANTALLA DE EJECUCIÓN 20
6.1. Evidencia del sistema en funcionamiento 20
6.2. Comparativa de eficiencia entre estrategias 24
7. CONCLUSIONES 27
7.1. Análisis y cierre 27
7.2. Propuestas de mejora 27
8. REFERENCIAS BIBLIOGRÁFICAS 28
9. APÉNDICES / ANEXOS 29
Anexo A. Instalación y ejecución 29
Anexo B. Mapas de prueba y rutas de A* 29
Anexo C. Código completo de los algoritmos 32
C.1. simulador/algoritmos/base.py 32

C.2. simulador/algoritmos/no_informados.py (BFS, DFS, UCS) 33
C.3. simulador/algoritmos/informados.py (Greedy, A*, Hill Climbing) 35

Índice de figuras

Figura 1. Diagrama de caso de uso del simulador de escape de incendios. 8
Figura 2. Diagrama de componentes del sistema. 12
Figura 3. Espacio de estados del mapa Escuela modelado como grafo, con la ruta óptima de A*. 13
Figura 4. Fragmento del espacio de estados con costos de transición y valores g, h y f de A*. 14
Figura 5. Pestaña Simulador: A* en el mapa Oficina desde (1, 1), con la ruta, los nodos expandidos y las métricas. 20
Figura 6. Pestaña Comparar algoritmos: tabla de métricas de los 6 algoritmos sobre el mismo mapa e inicio. 21
Figura 7. Gráficas comparativas de costo, nodos expandidos, tiempo y memoria. 21
Figura 8. Ruta y nodos expandidos de cada algoritmo. Hill Climbing aparece como «no llegó». 22
Figura 9. Pestaña Experimento completo: promedios de las 90 corridas. 22
Figura 10. Pestaña Memoria, primera consulta desde (8, 1): el agente RAZONA (ejecuta A* y guarda la ruta). 23
Figura 11. Pestaña Memoria, segunda consulta: el agente RECUERDA la ruta con 0 nodos expandidos. 23
Figura 12. Salida de consola: 135 pruebas aprobadas y resumen del experimento de 90 corridas. 24
Figura 13. Nodos expandidos promedio por algoritmo (esfuerzo de cálculo). 25
Figura 14. Exceso de costo sobre la ruta óptima (0 % = óptima). 25
Figura 15. Porcentaje de corridas que llegaron a una salida. 26
Figura 16. Mapa Oficina con las rutas de A* desde sus 5 puntos de inicio. 29
Figura 17. Mapa Escuela con las rutas de A* desde sus 5 puntos de inicio. 30
Figura 18. Mapa Bodega con las rutas de A* desde sus 5 puntos de inicio 31

Índice de tablas

Tabla 1. Formulación formal del problema de búsqueda 7
Tabla 2. Símbolos de la cuadrícula y costo de cada celda 8
Tabla 3. Descripción de los casos de uso 8
Tabla 4. Matriz PEAS (Rendimiento, Ambiente, Actuadores, Sensores) del agente evacuador 10
Tabla 5. Clasificación del ambiente de trabajo 11
Tabla 6. Componentes y responsabilidades 12
Tabla 7. Estructuras de datos utilizadas 14
Tabla 8. Las seis estrategias implementadas 15
Tabla 9. Resultados promedio por algoritmo (90 corridas) 24
Tabla 10. Nodos expandidos promedio por mapa 26
Tabla 11. Puntos de inicio por mapa 31

1. RESUMEN

1.1. Síntesis del proyecto

Problema. Durante un incendio, la ruta más corta hacia una salida no siempre es la más segura: atravesar zonas con humo aumenta el riesgo para la persona. Este proyecto modela un edificio de 20 × 20 celdas con muros, fuego, humo y varias salidas, y plantea la evacuación como un problema de búsqueda en un espacio de estados con costos.

Objetivo. Diseñar un agente inteligente que encuentre la ruta de evacuación de menor costo usando A* como algoritmo principal, y comparar su desempeño contra otras cinco estrategias (BFS, DFS, UCS, Greedy Best-First y Hill Climbing), todas programadas sin librerías de búsqueda.

Tipo de agente. Agente basado en metas (objetivos) que resuelve problemas por búsqueda. Percibe el entorno completo, planea la secuencia de movimientos antes de actuar e incorpora un componente de memoria que guarda las rutas ya calculadas (memoización).

Resultados clave. En 90 corridas (3 mapas × 5 puntos de inicio × 6 algoritmos), A* encontró la ruta de costo mínimo en el 100 % de los casos, igual que UCS, pero expandiendo en promedio un 55 % menos de nodos (34.8 contra 76.7). BFS obtuvo rutas con menos pasos, pero un 20.6 % más costosas por cruzar el humo; DFS produjo rutas 4.1 veces más caras que la óptima; Greedy fue el que menos nodos expandió (21.9), pero solo fue óptimo en 5 de 15 casos; y Hill Climbing quedó atrapado en mínimos locales en 13 de 15 casos. La memoización respondió 16 de 30 consultas sin buscar, incluida una ruta que nunca se había calculado de forma directa.

Palabras clave: agente basado en metas, A*, búsqueda heurística, distancia Manhattan, evacuación, espacio de estados, memoización.

2. INTRODUCCIÓN

2.1. Contexto teórico

En Inteligencia Artificial, un problema de búsqueda se define por un estado inicial, un conjunto de acciones, un modelo de transición, una prueba de meta y una función de costo de camino (Russell y Norvig, 2021). Un algoritmo de búsqueda construye un árbol de búsqueda a partir del estado inicial y decide, en cada paso, qué nodo de la frontera expandir. Esa decisión es lo que distingue a las estrategias.

Búsqueda ciega (no informada). No dispone de información sobre la ubicación de la meta; solo distingue entre un estado meta y uno que no lo es.

- **BFS** expande por niveles con una cola FIFO; es completa y óptima cuando todos los pasos cuestan lo mismo.
- **DFS** profundiza primero con una pila LIFO; usa poca memoria, pero no es óptima.
- **UCS** expande el nodo de menor costo acumulado g(n) con una cola de prioridad; es óptima con costos positivos y equivale al algoritmo de Dijkstra (1959) con parada temprana.

Búsqueda heurística (informada). Usa una función h(n) que estima el costo desde n hasta la meta.

- **Greedy Best-First** ordena la frontera solo por h(n).
- **A*** ordena por f(n) = g(n) + h(n) y es óptimo cuando h es admisible, es decir, cuando nunca sobreestima el costo real (Hart, Nilsson y Raphael, 1968).
- **Hill Climbing** es una búsqueda local que solo avanza hacia el vecino que mejora h; no guarda frontera y puede quedar atrapada en mínimos locales.

Las estrategias se comparan con cuatro criterios: completitud (¿siempre encuentra solución si existe?), optimalidad (¿encuentra la de menor costo?), complejidad temporal (nodos expandidos) y complejidad espacial (tamaño de la frontera y memoria).

2.2. Planteamiento del problema

El escenario es un edificio representado por una cuadrícula de 20 × 20 celdas. Cada celda puede ser pasillo libre, humo, muro, fuego o salida. El fuego no se propaga durante la simulación. Una persona parte de una celda y debe llegar a cualquiera de las salidas con el menor costo, entendido como la exposición acumulada al humo.

Tabla 1. Formulación formal del problema de búsqueda

Elemento	Definición en el sistema
Estados (S)	Posiciones (fila, columna) transitables: 0 ≤ fila, columna ≤ 19 y la celda no es muro ni fuego.
Estado inicial (s_0)	Uno de los 5 puntos de inicio definidos por mapa, o uno elegido en la interfaz.
Acciones A(s)	{arriba, abajo, izquierda, derecha}, restringidas a vecinos dentro del mapa y transitables.
Modelo de transición	T((f, c), arriba) = (f−1, c); abajo = (f+1, c); izquierda = (f, c−1); derecha = (f, c+1).
Prueba de meta	La celda del estado es una salida 'S'. Sirve cualquiera de las salidas del mapa.
Costo de paso c(s, a, s')	Costo de la celda destino s': 1 si es libre o salida, 5 si tiene humo.

Elemento	Definición en el sistema
Costo de camino g(n)	Suma de los costos de paso desde s_0 hasta n.
Solución óptima	Camino de s_0 a una salida con g mínimo.

Tabla 2. Símbolos de la cuadrícula y costo de cada celda

Símbolo	Significado	Transitable	Costo de entrar
.	Pasillo libre	Sí	1
H	Humo	Sí	5
S	Salida (meta)	Sí	1
#	Muro	No	—
F	Fuego (estático)	No	—

2.3. Diagrama de caso de uso

El sistema tiene dos actores. El usuario (estudiante o evaluador) configura el escenario y pide las comparaciones. El agente evacuador ejecuta la búsqueda, consulta su memoria y calcula las métricas.

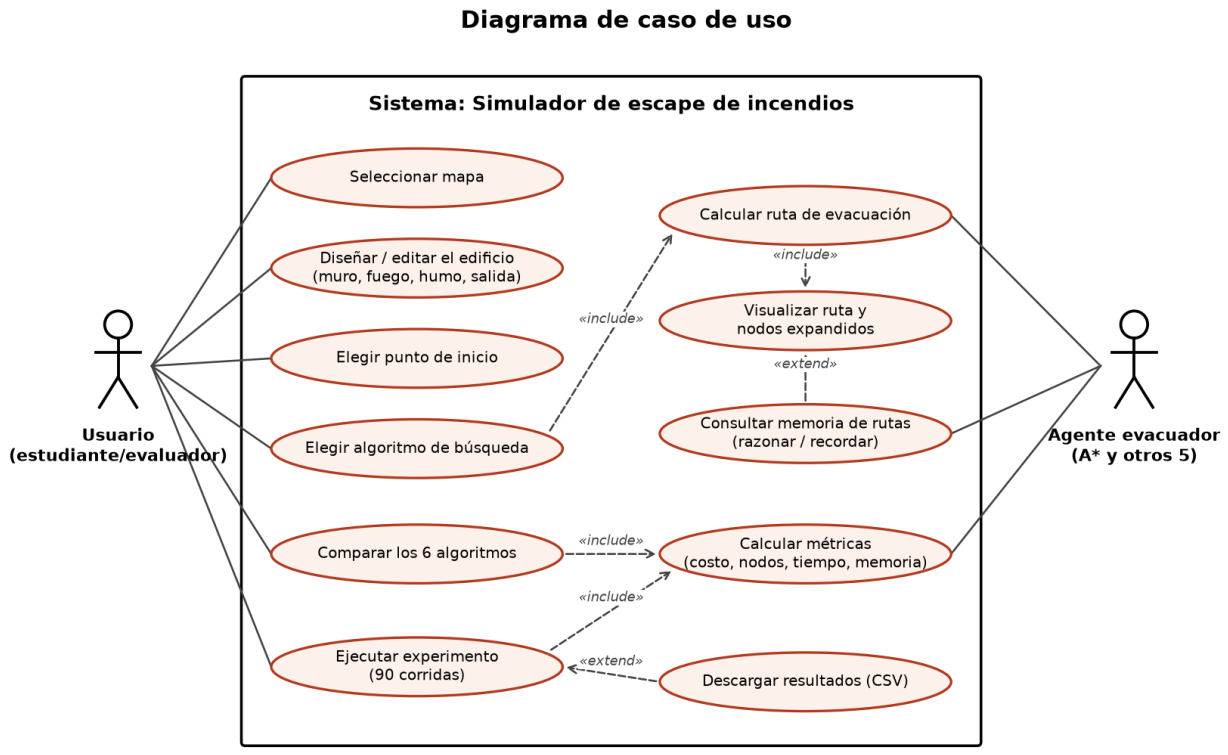


Figura 1. Diagrama de caso de uso del simulador de escape de incendios.

Tabla 3. Descripción de los casos de uso

Caso de uso	Actor	Descripción
Seleccionar mapa	Usuario	Elige entre Oficina, Escuela y Bodega.

Caso de uso	Actor	Descripción
Diseñar / editar el edificio	Usuario	Pinta celdas (muro, fuego, humo, salida, libre) o edita el mapa como tabla.
Elegir punto de inicio	Usuario	Uno de los 5 inicios del mapa o una celda personalizada.
Elegir algoritmo	Usuario	Selecciona uno de los 6 algoritmos; incluye «Calcular ruta».
Calcular ruta de evacuación	Agente	Ejecuta la búsqueda y reconstruye el camino; incluye «Visualizar ruta».
Consultar memoria de rutas	Agente	Extiende la visualización: responde desde memoria si ya conoce la ruta.
Comparar los 6 algoritmos	Usuario	Corre todos sobre el mismo mapa e inicio; incluye «Calcular métricas».
Ejecutar experimento	Usuario	Corre las 90 combinaciones y muestra promedios y gráficas.
Descargar resultados	Usuario	Extiende el experimento: exporta el CSV con las 90 corridas.

3. DESARROLLO

3.1. Arquitectura del agente inteligente

3.1.1. Identificación y clasificación del tipo de agente

Clasificación: agente basado en metas (objetivos) que resuelve problemas mediante búsqueda, con un componente de memoria de experiencias.

Se descartaron los otros tipos de agente por las siguientes razones:

- **No es un agente reactivo simple:** no responde con reglas condición-acción a la percepción inmediata. Antes de moverse, planea la secuencia completa de acciones hasta la meta.
- **No es solo basado en modelo:** además de tener un modelo del mundo (la cuadrícula y el modelo de transición), evalúa secuencias futuras de acciones en función de una meta explícita: llegar a una celda 'S'.
- **Es basado en metas:** la prueba de meta es_salida() guía la búsqueda, y la función de costo g(n) le permite preferir las rutas con menos exposición al humo. Esto acerca su comportamiento al de un agente basado en utilidad, porque no se conforma con cualquier ruta: busca la de menor costo.
- **Componente de aprendizaje:** el módulo MemoriaRutas guarda cada ruta calculada y, en el caso de A* y UCS, también todas sus sub-rutas óptimas. Ante una consulta repetida, el agente responde sin buscar. Así se distingue cuándo el sistema razona (ejecuta la búsqueda) y cuándo aprende (reutiliza la experiencia guardada).

3.1.2. Matriz de diagnóstico del modelo PEAS

Tabla 4. Matriz PEAS (Rendimiento, Ambiente, Actuadores, Sensores) del agente evacuador

Componente	Descripción	Implementación en el código
P – Medida de rendimiento	Llegar a una salida (éxito); minimizar el costo de la ruta, es decir, la exposición al humo; en segundo lugar, minimizar los pasos, los nodos expandidos, el tiempo y la memoria.	Resultado.exito, costo, pasos, nodos_expandidos; metricas.medir() (tiempo y memoria)
E – Ambiente	Edificio de 20 × 20 celdas con pasillos, humo (costo 5), muros y fuego (bloqueados) y de 2 a 3 salidas. Tres escenarios: Oficina, Escuela y Bodega.	Clase Cuadricula; simulador/mapas.py
A – Actuadores	Movimiento de una celda en 4 direcciones: arriba, abajo, izquierda y derecha. El plan final es la secuencia de movimientos de la ruta.	MOVIMIENTOS; Resultado.ruta; reconstruir_ruta()
S – Sensores	Percepción de la posición actual, de las celdas vecinas transitables, del costo de cada celda, de si la celda es una salida y de la distancia estimada a la salida más cercana.	vecinos(), transitable(), costo(), es_salida(), heuristica()

3.2. Propiedades del entorno de búsqueda

Tabla 5. Clasificación del ambiente de trabajo

Propiedad	Clasificación	Justificación
Observabilidad	**Totalmente observable**	El agente conoce el mapa completo: muros, fuego, humo y salidas.
Determinismo	**Determinista**	Cada acción lleva siempre a la misma celda y con el mismo costo; no hay azar.
Dinámica	**Estático**	El fuego y el humo no cambian mientras el agente planea ni mientras se mueve.
Espacio	**Discreto**	Hay un número finito de estados (máximo 400 celdas) y de acciones (4).
Episodios	**Secuencial**	Cada movimiento condiciona los siguientes; el costo se acumula a lo largo de la ruta.
Agentes	**Agente único**	Solo evacúa una persona y no hay otros agentes que compitan.
Conocimiento	**Conocido**	Las reglas de transición y los costos son conocidos de antemano.

Influencia en el diseño. Como el entorno es totalmente observable, determinista, estático y discreto, el agente puede calcular el plan completo antes de moverse, sin necesidad de volver a planear durante la ejecución. Esto permite usar búsqueda clásica en grafos. Si el fuego se propagara (entorno dinámico), habría que volver a planear la ruta en cada paso.

3.4. Diagrama de componentes del caso a desarrollar

El sistema se organiza en tres capas. La capa de presentación incluye la interfaz Streamlit y el script de experimentos. La capa de servicios incluye las métricas, la memoria de rutas y la visualización. La capa del agente y su entorno incluye los algoritmos, la cuadrícula y los mapas. Los algoritmos no dependen de la interfaz, por lo que se pueden probar de forma aislada.

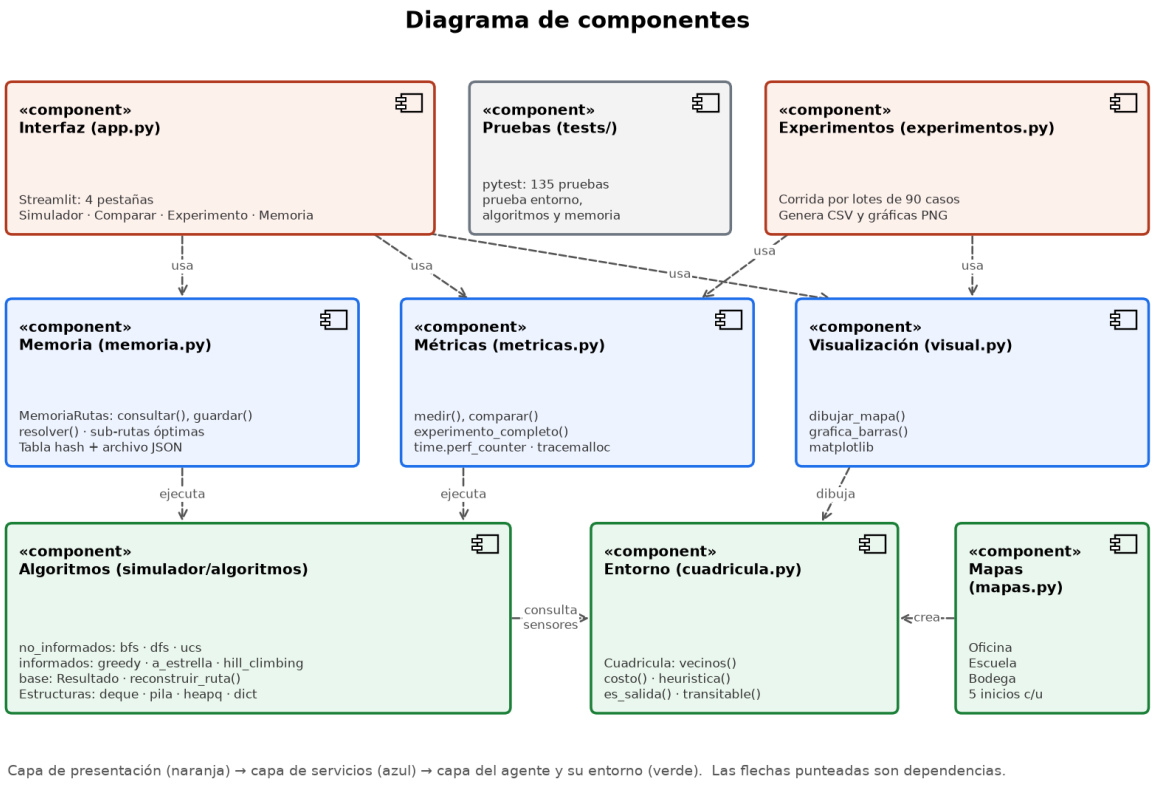


Figura 2. Diagrama de componentes del sistema.

Tabla 6. Componentes y responsabilidades

Componente	Archivo	Responsabilidad
Interfaz	app.py	Edición del mapa, selección de algoritmo e inicio, visualización y comparación
Experimentos	experimentos.py	Ejecuta las 90 corridas y genera el CSV y las gráficas
Métricas	simulador/metricas.py	Mide tiempo (perf_counter) y memoria (tracemalloc)
Memoria	simulador/memoria.py	Memoización de rutas y sub-rutas en tabla hash y archivo JSON
Visualización	simulador/visual.py	Dibuja el mapa, las celdas expandidas, la ruta y las gráficas
Algoritmos	simulador/algoritmos/	BFS, DFS, UCS, Greedy, A*, Hill Climbing y reconstrucción de rutas
Entorno	simulador/cuadricula.py	Representación del edificio, sensores, costos y heurística
Mapas	simulador/mapas.py	Los 3 escenarios con sus 5 puntos de inicio
Pruebas	tests/test_simulador.py	135 pruebas automáticas (pytest)

3.5. Abstracción del espacio de estados

Modelo matemático. El entorno se modela como un grafo dirigido y ponderado G = (V, E, w):

- **V** = {(f, c) | 0 ≤ f, c < 20, celda(f, c) ∉ {#, F}} son los nodos (estados).

- **E** = {((f, c), (f', c')) | |f − f'| + |c − c'| = 1, ambos en V} son las aristas (acciones).
- **w(u, v)** = costo(v) ∈ {1, 5} es el peso, que depende solo de la celda destino. Por eso el grafo es dirigido: entrar al humo cuesta 5, pero salir de él hacia una celda libre cuesta 1.

El mapa Escuela, por ejemplo, tiene 232 nodos y 329 pares de celdas vecinas, es decir, 658 aristas dirigidas. La Figura siguiente muestra ese grafo con la ruta óptima de A* desde el inicio (1, 9).

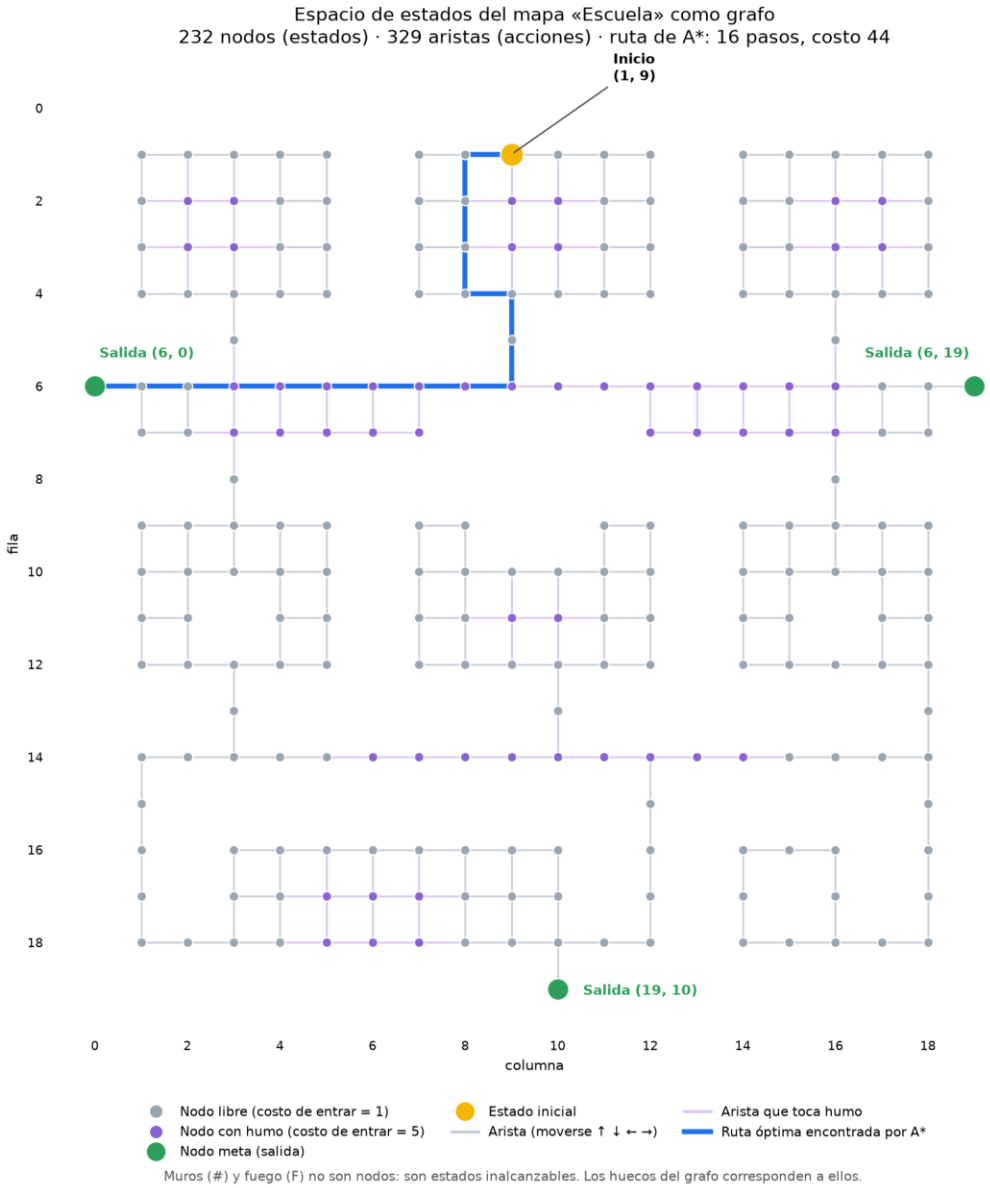


Figura 3. Espacio de estados del mapa Escuela modelado como grafo, con la ruta óptima de A*.

Para explicar los pesos y los valores que calcula A*, la siguiente figura muestra un fragmento pequeño. La ruta superior tiene 4 pasos, pero cruza dos celdas de humo (costo 12). La ruta inferior tiene 8 pasos y cuesta 8. A* elige la inferior porque sus nodos tienen f = 8, menor que el f = 12 de la ruta con humo.

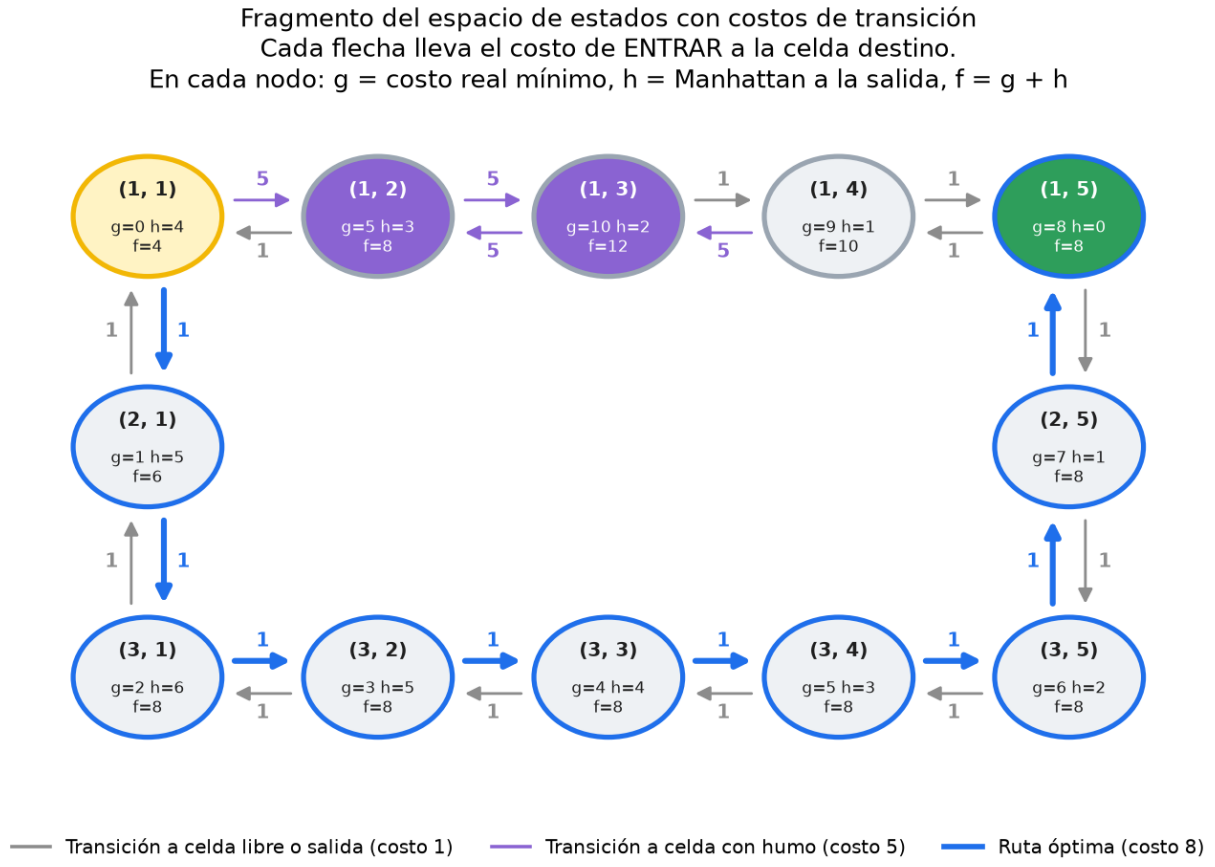


Figura 4. Fragmento del espacio de estados con costos de transición y valores g, h y f de A*.

Matriz del entorno. Internamente, la cuadrícula se guarda como una matriz de caracteres (list[list[str]]) de 20 × 20. Las aristas no se almacenan: se generan bajo demanda con vecinos(), que recorre los 4 desplazamientos y descarta las celdas fuera del mapa, los muros y el fuego. Así se evita construir una lista de adyacencia explícita.

Tabla 7. Estructuras de datos utilizadas

Estructura	Implementación en Python	Uso en el sistema
Matriz	list[list[str]]	Representación del edificio (Cuadricula.celdas)
Cola FIFO	collections.deque	Frontera de BFS (popleft / append en O(1))
Pila LIFO	list (append / pop)	Frontera de DFS
Cola de prioridad	heapq (montículo binario)	Frontera de UCS (por g), Greedy (por h) y A* (por f, h, orden de llegada)
Tabla hash	dict	Padres de cada nodo, costos g y memoria de rutas (clave: huella|algoritmo|inicio)
Conjunto	set	Nodos cerrados (ya expandidos) para no repetir estados
Árbol de búsqueda	dict padres: hijo → padre	Árbol implícito que permite el rastreo inverso del camino

4. ALGORITMO DE BÚSQUEDA UTILIZADO

4.1. Descripción de la estrategia seleccionada

Estrategia principal: A*. A* mantiene una frontera ordenada por f(n) = g(n) + h(n) y siempre expande el nodo con menor f. Combina la garantía de optimalidad de UCS (por usar g) con la dirección hacia la meta de Greedy (por usar h).

Ciclo percepción-acción del agente con A*:

1. **Percibir:** toma de la frontera el nodo n con menor f y consulta el sensor es_salida(n).
2. **Si n es una salida:** termina y ejecuta el rastreo inverso (reconstruir_ruta) para obtener la secuencia de acciones.
3. **Si no lo es:** lo marca como cerrado y consulta el sensor vecinos(n), que devuelve las celdas a las que puede moverse (sus actuadores posibles).
4. **Para cada vecino v:** lee costo(v) y calcula g' = g(n) + costo(v). Si g' mejora el g conocido de v, guarda padre[v] = n, calcula h(v) con el sensor heuristica() y mete v a la frontera con f = g' + h(v).
5. **Repite** hasta encontrar una salida o vaciar la frontera, en cuyo caso no hay ruta.

Desempates. Si dos nodos tienen el mismo f, se prefiere el de menor h (el que está más cerca de salir). Si también empatan en h, se usa el orden de llegada. Esto hace que el algoritmo sea determinista.

Convenciones comunes a los 6 algoritmos. Para que la comparación sea justa, todos siguen las mismas reglas: la prueba de meta se hace al expandir, los vecinos se generan en el orden arriba, abajo, izquierda y derecha, y todos son búsquedas en grafo (salvo Hill Climbing, que no guarda historial).

Tabla 8. Las seis estrategias implementadas

Algoritmo	Tipo	Frontera	Criterio	Completo	Óptimo
BFS	Ciega	Cola FIFO	Nivel (pasos)	Sí	No*
DFS	Ciega	Pila LIFO	Profundidad	Sí	No
UCS	Ciega	Cola de prioridad	g(n)	Sí	Sí
Greedy	Heurística	Cola de prioridad	h(n)	Sí	No
A*	**Heurística**	**Cola de prioridad**	**g(n) + h(n)**	**Sí**	**Sí**
Hill Climbing	Local	Ninguna	Mejor vecino por h	No	No

** BFS solo es óptimo en número de pasos, no en costo, porque ignora el costo del humo.*

4.2. Formulación de costos

Costo real acumulado g(n). Es la suma de los costos de las celdas a las que se entró desde el inicio:

$$g(n) = \Sigma\ costo(s_i),\ \ con\ \ costo(s) = 1\ si\ s \in \{., S\}\ \ y\ \ costo(s) = 5\ si\ s = H$$

Función heurística h(n). Es la distancia Manhattan a la salida más cercana:

$$h(n) = min\ \ (|f_n - f_s| + |c_n - c_s|),\ \ para\ toda\ salida\ s$$

Admisibilidad. Con movimientos en 4 direcciones, se necesitan al menos |Δf| + |Δc| pasos para llegar a una salida, y cada paso cuesta como mínimo 1. Por lo tanto, h(n) ≤ h*(n), donde h* es el costo real óptimo: la

heurística nunca sobreestima. La prueba automática test_heuristica_es_admisible lo verifica en todas las celdas transitables de los tres mapas.

Consistencia. Al moverse a un vecino n', la distancia Manhattan cambia como mucho en 1, y el paso cuesta $c(n, n') \geq 1$. Entonces $h(n) \leq c(n, n') + h(n')$. Gracias a esto, A* nunca necesita volver a abrir un nodo ya cerrado.

Vínculo con la medida de rendimiento. La medida de rendimiento del agente es minimizar la exposición al humo, lo cual equivale a minimizar g. Por eso g usa exactamente los mismos costos que el rendimiento (1 y 5), y h es una cota inferior optimista de ese mismo costo. Así, f(n) estima el costo total de la mejor ruta que pasa por n, y A* devuelve la ruta que maximiza el rendimiento.

Calidad de la heurística. Cuando la ruta óptima atraviesa humo, el costo real es mucho mayor que la distancia Manhattan y h pierde precisión. Por eso A* ahorra un 73 % de nodos frente a UCS en la Oficina (poco humo en las rutas), pero solo un 24 % en la Escuela, cuyo pasillo central está lleno de humo (ver sección 6.2).

5. CÓDIGO FUENTE

5.1. Implementación de clases y funciones

El código está escrito en Python 3, organizado por módulos y documentado con docstrings en español. A continuación se muestran los fragmentos centrales, extraídos tal cual del repositorio. El código completo de los algoritmos está en el Anexo C.

a) Sensores del agente: clase Cuadricula

Estos métodos son la percepción del agente: qué celdas vecinas puede pisar, cuánto cuesta entrar a cada una, si una celda es salida y qué tan lejos está la salida más cercana.

Archivo: simulador/cuadricula.py (líneas 96–97)

```
96      def transitable(self, pos: Posicion) -> bool:
97          return self.dentro(pos) and self.simbolo(pos) not in BLOQUEADAS
```

Archivo: simulador/cuadricula.py (líneas 99–100)

```
 99      def es_salida(self, pos: Posicion) -> bool:
100          return self.simbolo(pos) == SALIDA
```

Archivo: simulador/cuadricula.py (líneas 102–104)

```
102      def costo(self, pos: Posicion) -> int:
103          """Costo de ENTRAR a la celda `pos`."""
104          return COSTOS[self.simbolo(pos)]
```

Archivo: simulador/cuadricula.py (líneas 106–116)

```
106      def vecinos(self, pos: Posicion) -> list[Posicion]:
107          """Celdas transitables alcanzables en un paso, en el orden de MOVIMIENTOS."""
108          f, c = pos
109          resultado = []
110          for df, dc in MOVIMIENTOS:
111              nueva = (f + df, c + dc)
112              if self.transitable(nueva):
113                  resultado.append(nueva)
114          return resultado
115
116      # --- Heurística ----------------------------------------------------------
```

Archivo: simulador/cuadricula.py (líneas 117–131)

```
117      def heuristica(self, pos: Posicion) -> int:
118          """
119          Distancia Manhattan a la salida MÁS CERCANA.
120
121          Es admisible: cada paso cuesta al menos 1 y necesitamos al menos
122          |df| + |dc| pasos para llegar a una salida, así que nunca sobreestima.
123          También es consistente: al movernos un paso, h cambia como mucho en 1,
124          y ese paso cuesta >= 1.
125          """
126          if not self.salidas:
127              return 0
128          f, c = pos
129          return min(abs(f - sf) + abs(c - sc) for sf, sc in self.salidas)
130
131      # --- Utilidades ----------------------------------------------------------
```

b) Actuadores: conjunto de movimientos

Archivo: simulador/cuadricula.py (líneas 37–44)

```
37  MOVIMIENTOS = (
38      (-1, 0),  # arriba
39      (1, 0),   # abajo
40      (0, -1),  # izquierda
```

```
41      (0, 1),   # derecha
42  )
43
44  Posicion = tuple[int, int]
```

*c) Expansión de nodos: algoritmo A**

Archivo: simulador/algoritmos/informados.py (líneas 64–115)

```
64  def a_estrella(mapa: Cuadricula, inicio: Posicion) -> Resultado:
65      """
66      A* (algoritmo principal del proyecto).
67
68      Ordena la frontera por f(n) = g(n) + h(n):
69        g(n) = costo real desde el inicio (cuenta el humo como 5),
70        h(n) = Manhattan a la salida más cercana (estimación optimista).
71
72      Como h nunca sobreestima (admisible) y es consistente, la primera salida
73      que se expande es la de menor costo, igual que UCS, pero explorando
74      menos celdas porque h "empuja" la búsqueda hacia las salidas.
75      En empate de f se prefiere la celda con menor h (la más cercana a salir).
76
77      Completa: sí.  Óptima en costo: sí.
78      """
79      nombre = "A*"
80      if error := validar_inicio(mapa, inicio):
81          return Resultado(nombre, False, mensaje=error)
82
83      desempate = count()
84      h0 = mapa.heuristica(inicio)
85      frontera = [(h0, h0, next(desempate), inicio)]  # (f, h, orden de llegada, celda)
86      g: dict[Posicion, int] = {inicio: 0}
87      padres: dict[Posicion, Posicion | None] = {inicio: None}
88      cerrados: set[Posicion] = set()
89      expandidos = generados = 0
90      max_frontera = 1
91      orden: list[Posicion] = []
92
93      while frontera:
94          _, _, _, actual = heapq.heappop(frontera)
95          if actual in cerrados:
96              continue
97          cerrados.add(actual)
98          expandidos += 1
99          orden.append(actual)
100
101         if mapa.es_salida(actual):
102             return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)
103
104         for vecino in mapa.vecinos(actual):
105             nuevo_g = g[actual] + mapa.costo(vecino)
106             if vecino not in cerrados and nuevo_g < g.get(vecino, float("inf")):
107                 g[vecino] = nuevo_g
108                 padres[vecino] = actual
109                 h = mapa.heuristica(vecino)
110                 heapq.heappush(frontera, (nuevo_g + h, h, next(desempate), vecino))
111                 generados += 1
112         max_frontera = max(max_frontera, len(frontera))
113
114     return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
115                      max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")
```

d) Rastreo inverso (backtracking) del camino óptimo

Cada nodo guarda a su padre en un diccionario. Al llegar a la salida, se siguen esos punteros hacia atrás hasta el inicio y se invierte la lista para obtener la ruta en orden.

Archivo: simulador/algoritmos/base.py (líneas 71–79)

```
71  def reconstruir_ruta(padres: dict[Posicion, Posicion | None], meta: Posicion) -> list[Posicion]:
72      """Sigue los punteros a padre desde la meta hasta el inicio y voltea la lista."""
73      ruta = []
74      actual: Posicion | None = meta
```

```
75      while actual is not None:
76          ruta.append(actual)
77          actual = padres[actual]
78      ruta.reverse()
79      return ruta
```

e) Componente de aprendizaje: memoización

Archivo: simulador/memoria.py (líneas 93–105)

```
 93      def resolver(self, mapa: Cuadricula, algoritmo: str, inicio: Posicion) -> tuple[Resultado, bool]:
 94          """
 95          Devuelve (resultado, vino_de_memoria).
 96          Si ya lo sabe lo recuerda; si no, razona con el algoritmo y lo aprende.
 97          """
 98          recordado = self.consultar(mapa, algoritmo, inicio)
 99          if recordado is not None:
100              self.aciertos += 1
101              return recordado, True
102          self.fallos += 1
103          resultado = ALGORITMOS[algoritmo](mapa, inicio)
104          self.guardar(mapa, algoritmo, inicio, resultado)
105          return resultado, False
```

6. CAPTURA DE PANTALLA DE EJECUCIÓN

6.1. Evidencia del sistema en funcionamiento

La interfaz se ejecuta con «streamlit run app.py». La barra lateral permite elegir el mapa y el punto de inicio. El área principal tiene cuatro pestañas. En el mapa, la estrella marca el inicio, la línea de color es la ruta encontrada y los puntos transparentes son los nodos que el algoritmo expandió.

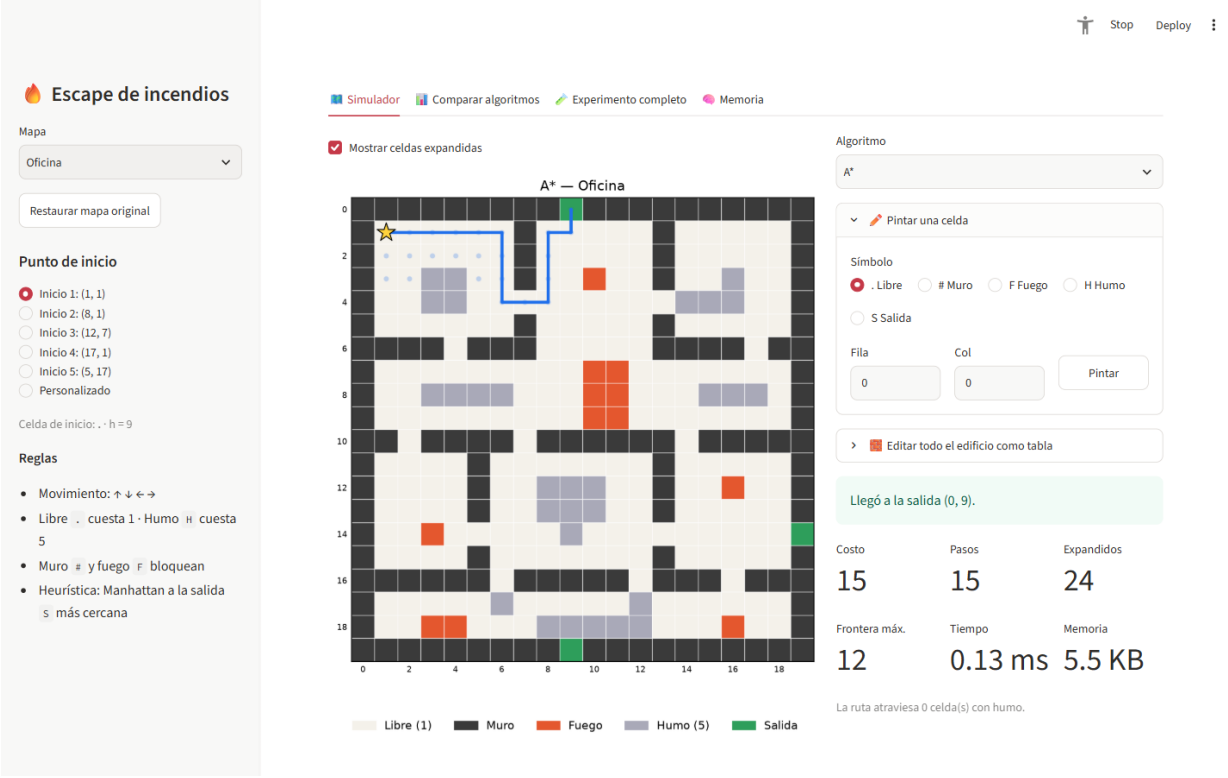


Figura 5. Pestaña Simulador: A* en el mapa Oficina desde (1, 1), con la ruta, los nodos expandidos y las métricas.

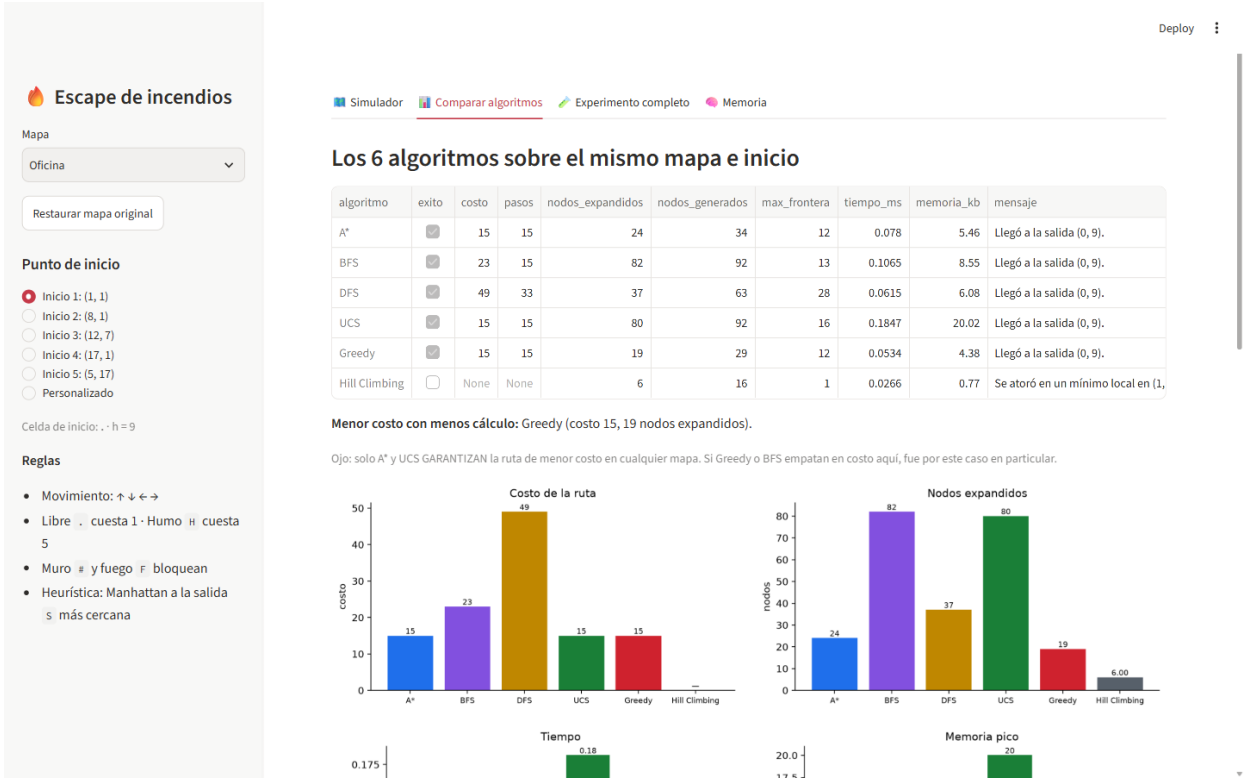


Figura 6. Pestaña Comparar algoritmos: tabla de métricas de los 6 algoritmos sobre el mismo mapa e inicio.

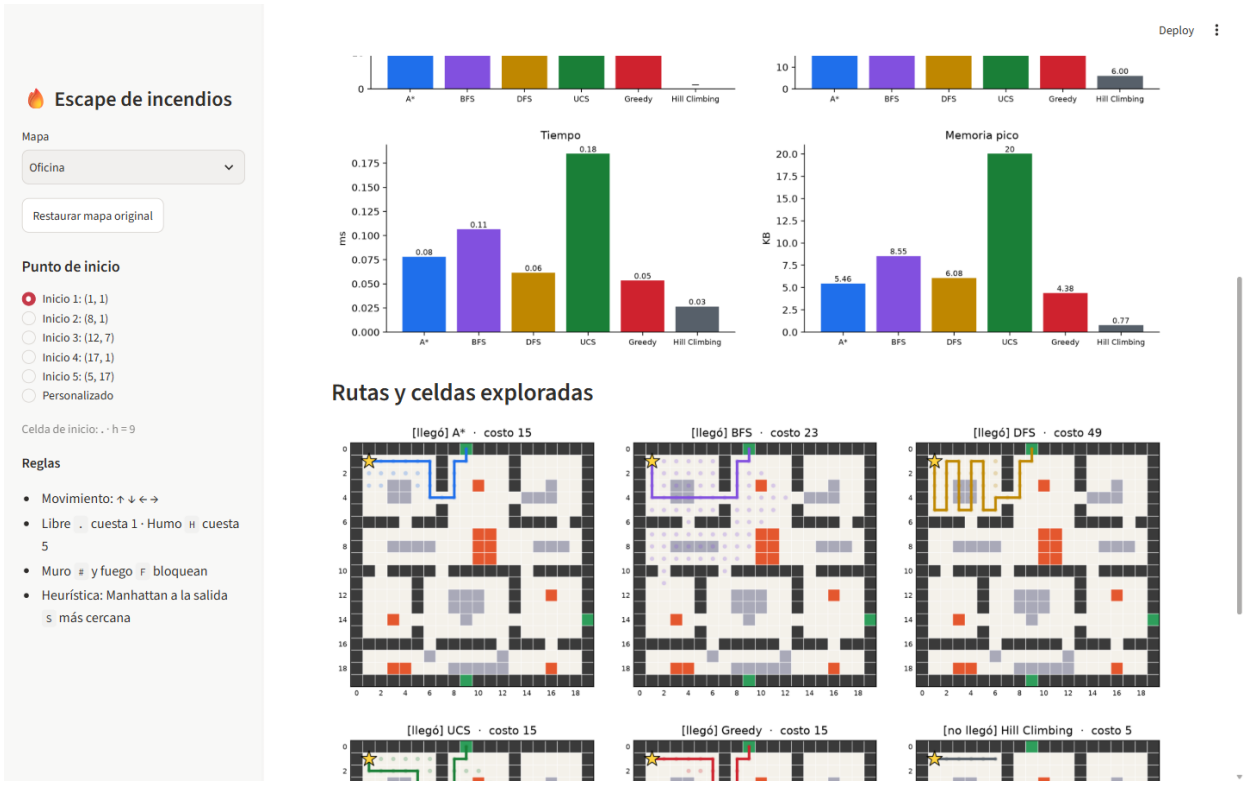


Figura 7. Gráficas comparativas de costo, nodos expandidos, tiempo y memoria.



Figura 8. Ruta y nodos expandidos de cada algoritmo. Hill Climbing aparece como «no llegó».



Figura 9. Pestaña Experimento completo: promedios de las 90 corridas.

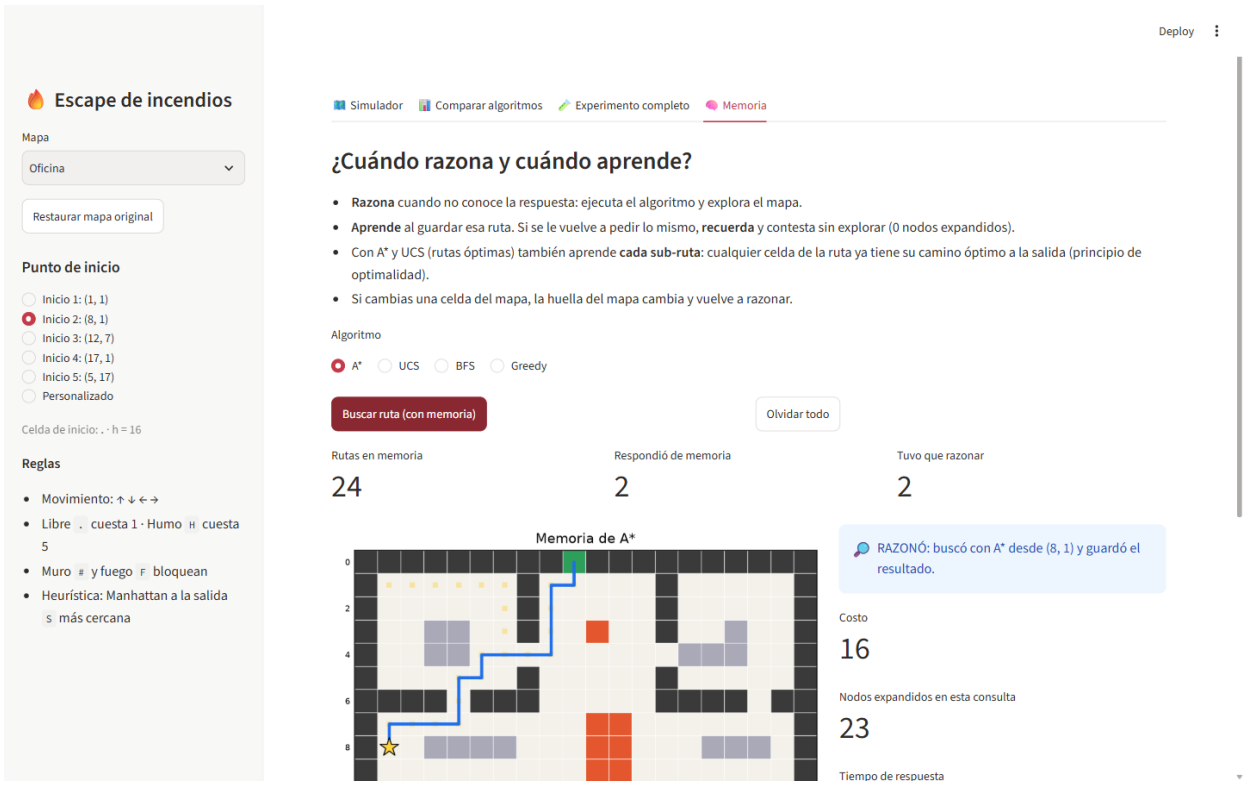


Figura 10. Pestaña Memoria, primera consulta desde (8, 1): el agente RAZONA (ejecuta A* y guarda la ruta).

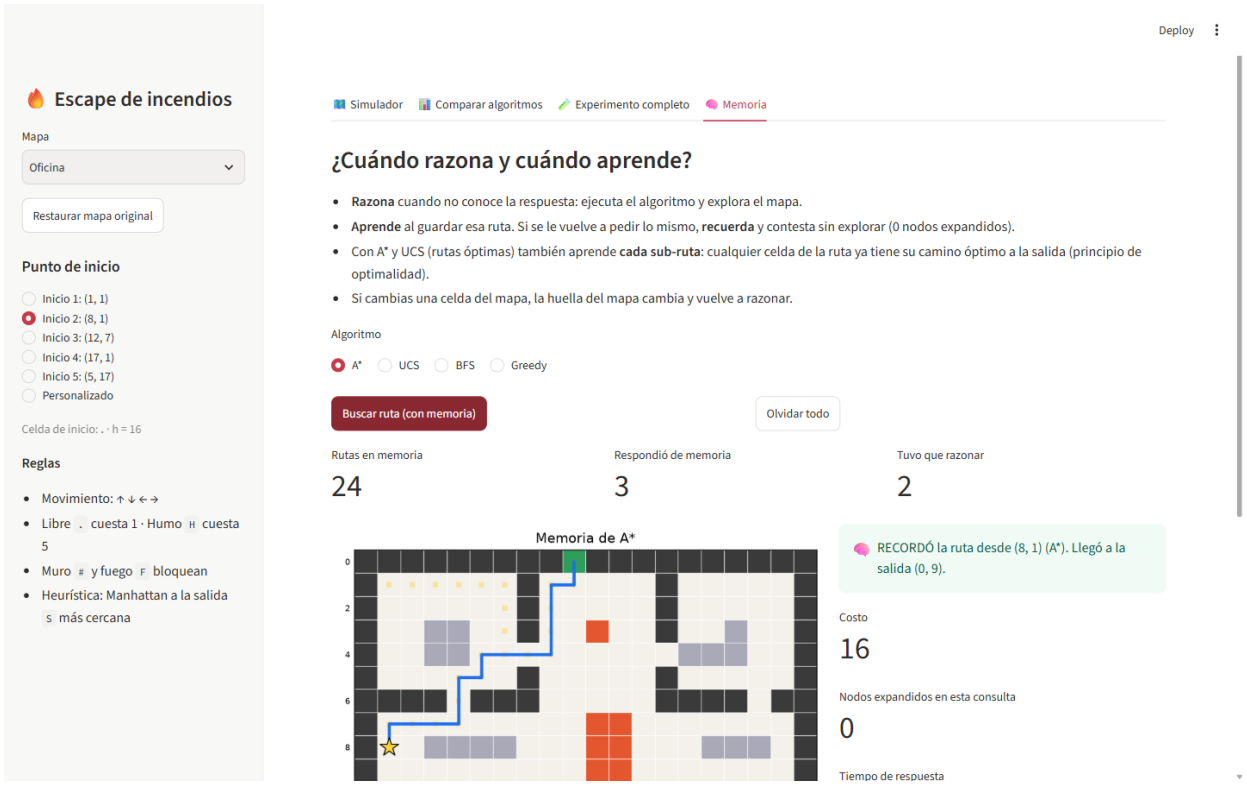


Figura 11. Pestaña Memoria, segunda consulta: el agente RECUERDA la ruta con 0 nodos expandidos.

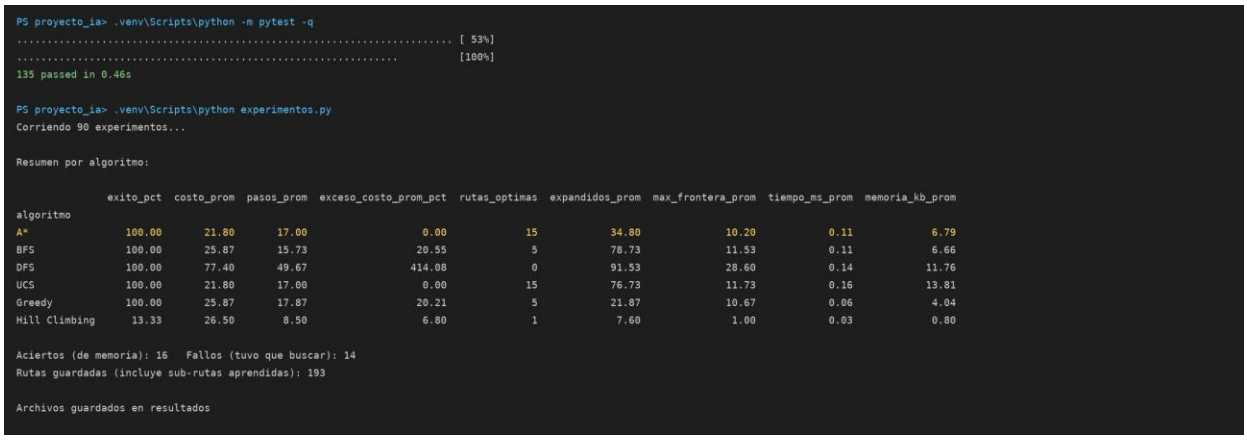


Figura 12. Salida de consola: 135 pruebas aprobadas y resumen del experimento de 90 corridas.

6.2. Comparativa de eficiencia entre estrategias

Los datos provienen de 90 corridas: 3 mapas × 5 puntos de inicio × 6 algoritmos. El tiempo es la mediana de 7 ejecuciones y la memoria es el pico medido con tracemalloc. El costo y los pasos solo promedian las corridas exitosas. El exceso de costo compara cada ruta con la óptima de UCS.

Tabla 9. Resultados promedio por algoritmo (90 corridas)

Algoritmo	Éxito	Costo	Pasos	Exceso	Óptimas	Expandidos	Frontera	Tiempo ms	Memoria KB
A*	100 %	21.8	17.0	0 %	15/15	34.8	10.2	0.11	6.8
BFS	100 %	25.9	15.7	20.6 %	5/15	78.7	11.5	0.11	6.7
DFS	100 %	77.4	49.7	414.1 %	0/15	91.5	28.6	0.14	11.8
UCS	100 %	21.8	17.0	0 %	15/15	76.7	11.7	0.16	13.8
Greedy	100 %	25.9	17.9	20.2 %	5/15	21.9	10.7	0.06	4.0
Hill Climbing	13.3 %	26.5*	8.5*	6.8 %*	1/15	7.6	1.0	0.03	0.8

** Promedio de las 2 corridas exitosas de Hill Climbing.*

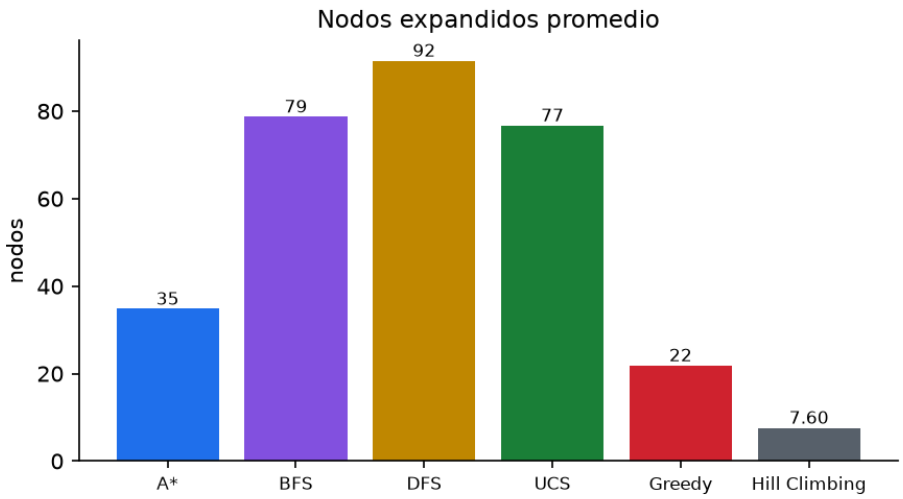


Figura 13. Nodos expandidos promedio por algoritmo (esfuerzo de cálculo).

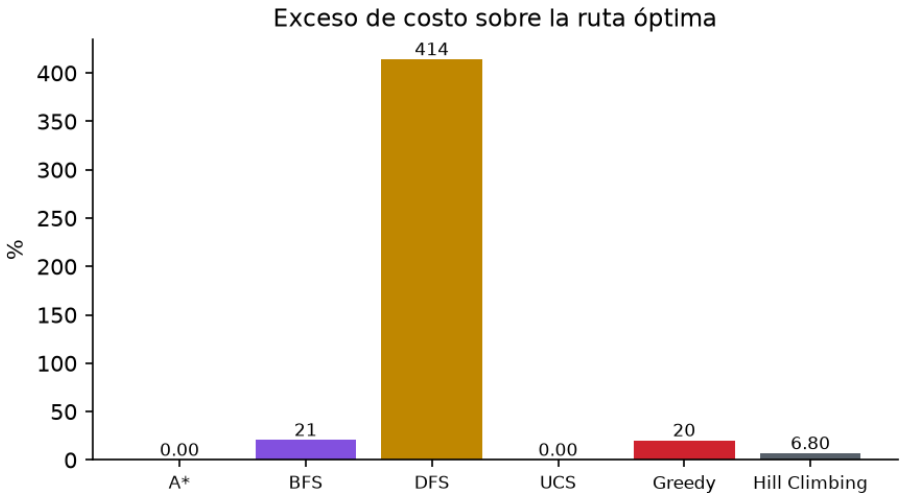


Figura 14. Exceso de costo sobre la ruta óptima (0 % = óptima).

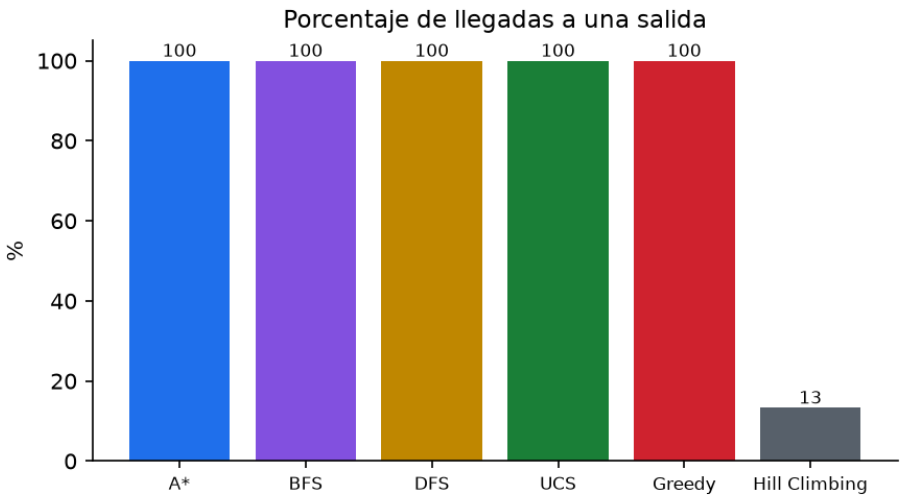


Figura 15. Porcentaje de corridas que llegaron a una salida.

Tabla 10. Nodos expandidos promedio por mapa

Mapa	A*	UCS	Ahorro de A* vs. UCS	Greedy	BFS	DFS
Oficina	28.8	105.6	73 %	20.8	93.8	54.6
Bodega	47.4	87.4	46 %	29.8	92.2	136.4
Escuela	28.2	37.2	24 %	15.0	50.2	83.6

Interpretación. A* es el único algoritmo que logra a la vez el costo mínimo y un número bajo de expansiones. Greedy expande todavía menos nodos, pero a cambio de rutas un 20 % más caras. Las diferencias de tiempo son de centésimas de milisegundo y están dominadas por el ruido del sistema operativo, así que el indicador más confiable del esfuerzo de cálculo son los nodos expandidos.

7. CONCLUSIONES

7.1. Análisis y cierre

Rendimiento computacional. A* cumplió el objetivo del proyecto: encontró la ruta de evacuación de costo mínimo en los 15 escenarios y expandió en promedio 34.8 nodos, frente a los 76.7 de UCS, con la mitad de memoria pico (6.8 KB contra 13.8 KB). Su costo computacional depende de la calidad de h: el ahorro va del 73 % en la Oficina al 24 % en la Escuela.

Influencia de las propiedades del entorno.

- **Totalmente observable y estático:** permitieron planear la ruta completa una sola vez. Si el fuego se propagara, el plan podría invalidarse en plena ejecución.
- **Determinista y discreto:** hicieron posible modelar el edificio como un grafo finito y garantizar la completitud de BFS, DFS, UCS, Greedy y A*.
- **Costos no uniformes (humo = 5):** son la razón por la que BFS deja de ser óptimo y por la que la heurística Manhattan pierde precisión en mapas con mucho humo.
- **Muros y callejones:** crean mínimos locales de la distancia Manhattan. Hill Climbing, que no puede empeorar temporalmente, quedó atrapado en el 87 % de los casos; por ejemplo, en la Bodega se detuvo a 3 celdas de la salida, con un muro de por medio.

Razonar y aprender. El agente razona cuando ejecuta la búsqueda y aprende cuando guarda el resultado. Al reutilizar sub-rutas óptimas (principio de optimalidad), la memoria respondió 16 de 30 consultas sin buscar, con 0 nodos expandidos.

7.2. Propuestas de mejora

6. **Fuego dinámico:** permitir que el fuego y el humo se propaguen en el tiempo y volver a planear la ruta en cada paso con A* repetido o D* Lite.
7. **Heurística más informada:** precalcular distancias reales con un Dijkstra inverso desde las salidas, o ponderar la distancia Manhattan con la densidad de humo cercana, cuidando que siga siendo admisible.
8. **Hill Climbing mejorado:** agregar movimientos laterales, reinicios aleatorios o recocido simulado para escapar de los mínimos locales.
9. **Múltiples personas:** simular varios agentes y la congestión en puertas y pasillos.
10. **Edición con el ratón:** permitir dibujar el edificio haciendo clic sobre la cuadrícula.
11. **Memoria con versión:** incluir una versión del algoritmo en la clave de la caché, para invalidarla automáticamente si cambia el código.

8. REFERENCIAS BIBLIOGRÁFICAS

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Hart, P. E., Nilsson, N. J., y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

ISO/IEC/IEEE. (2017). *ISO/IEC/IEEE 12207:2017. Systems and software engineering — Software life cycle processes*. International Organization for Standardization.

Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Addison-Wesley.

Python Software Foundation. (s. f.). *heapq — Heap queue algorithm*. Python 3 Documentation. https://docs.python.org/3/library/heapq.html

Russell, S., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

Streamlit Inc. (s. f.). *Streamlit documentation*. https://docs.streamlit.io

9. APÉNDICES / ANEXOS

Anexo A. Instalación y ejecución

Requisitos: Python 3.10 o superior. Los comandos son para Windows (PowerShell), desde la carpeta del proyecto:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m streamlit run app.py
.venv\Scripts\python experimentos.py
.venv\Scripts\python diagramas.py
.venv\Scripts\python -m pytest -v
```

La interfaz abre en http://localhost:8501. El código fuente completo está en github.com/luarmoro123/proyecto-ia.

Anexo B. Mapas de prueba y rutas de A*

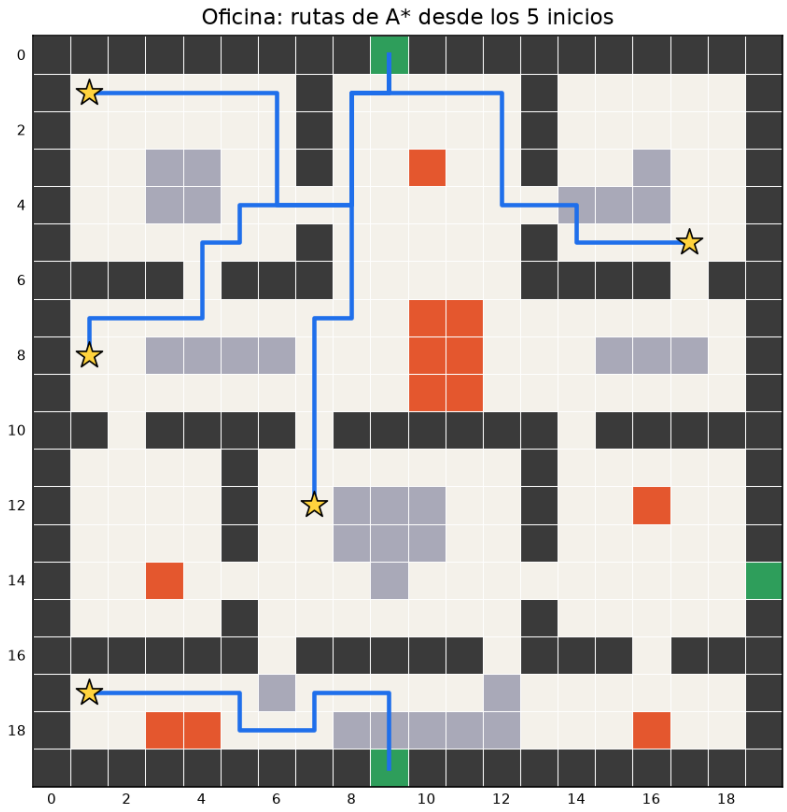


Figura 16. Mapa Oficina con las rutas de A* desde sus 5 puntos de inicio.

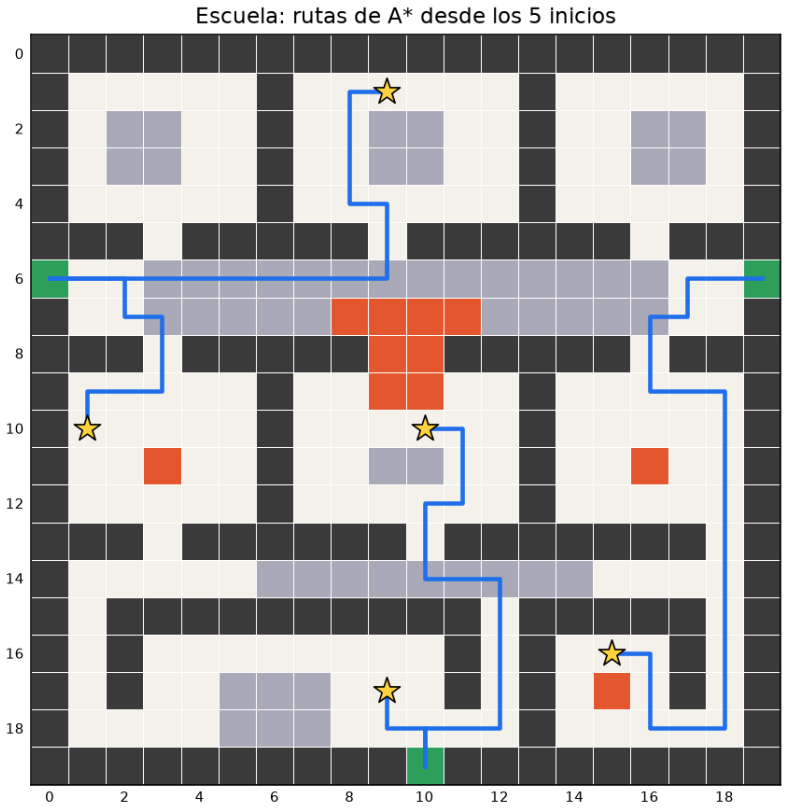


Figura 17. Mapa Escuela con las rutas de A* desde sus 5 puntos de inicio.

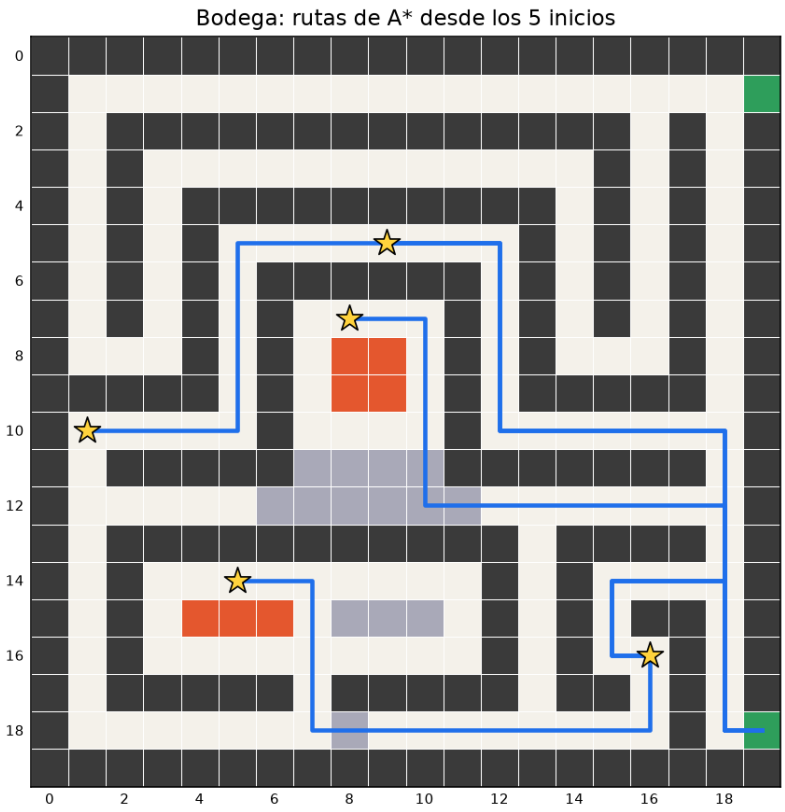


Figura 18. Mapa Bodega con las rutas de A* desde sus 5 puntos de inicio.

Tabla 11. Puntos de inicio por mapa

Mapa	Salidas	Puntos de inicio
Oficina	(0, 9), (14, 19), (19, 9)	(1, 1), (8, 1), (12, 7), (17, 1), (5, 17)
Escuela	(6, 0), (6, 19), (19, 10)	(1, 9), (10, 1), (10, 10), (17, 9), (16, 15)
Bodega	(1, 19), (18, 19)	(7, 8), (5, 9), (14, 5), (10, 1), (16, 16)

Anexo C. Código completo de los algoritmos

C.1. simulador/algoritmos/base.py

Archivo: simulador/algoritmos/base.py (líneas 1–114)

```
"""
Piezas comunes a todos los algoritmos de búsqueda.

Convenciones que siguen los 6 algoritmos (para que la comparación sea justa):

* La prueba de objetivo se hace al EXPANDIR un nodo (al sacarlo de la frontera),
  no al generarlo. Así "nodos expandidos" significa lo mismo en todos.
* Los vecinos se generan siempre en el mismo orden: arriba, abajo, izquierda, derecha.
* Los empates en colas de prioridad se rompen por orden de llegada (FIFO),
  usando un contador incremental.
* Todos son búsqueda en GRAFO: recuerdan qué celdas ya visitaron para no
  repetirlas (salvo Hill Climbing, que no guarda historial).
"""

from __future__ import annotations

from dataclasses import dataclass, field

from ..cuadricula import Cuadricula, Posicion


@dataclass
class Resultado:
    """Lo que devuelve cualquier algoritmo de búsqueda."""

    algoritmo: str
    exito: bool
    ruta: list[Posicion] = field(default_factory=list)
    costo: int = 0
    nodos_expandidos: int = 0
    nodos_generados: int = 0
    max_frontera: int = 0
    # Orden en el que se expandieron las celdas (para dibujarlas en la interfaz).
    orden_expansion: list[Posicion] = field(default_factory=list)
    mensaje: str = ""

    @property
    def pasos(self) -> int:
        """Número de movimientos de la ruta (celdas - 1)."""
        return max(len(self.ruta) - 1, 0)

    def a_dict(self) -> dict:
        """Versión serializable (se usa en la caché JSON)."""
        return {
            "algoritmo": self.algoritmo,
            "exito": self.exito,
            "ruta": [list(p) for p in self.ruta],
            "costo": self.costo,
            "nodos_expandidos": self.nodos_expandidos,
            "nodos_generados": self.nodos_generados,
            "max_frontera": self.max_frontera,
            "orden_expansion": [list(p) for p in self.orden_expansion],
            "mensaje": self.mensaje,
        }

    @classmethod
    def desde_dict(cls, d: dict) -> "Resultado":
        return cls(
            algoritmo=d["algoritmo"],
            exito=d["exito"],
            ruta=[tuple(p) for p in d["ruta"]],
            costo=d["costo"],
            nodos_expandidos=d["nodos_expandidos"],
            nodos_generados=d["nodos_generados"],
            max_frontera=d["max_frontera"],
            orden_expansion=[tuple(p) for p in d["orden_expansion"]],
            mensaje=d.get("mensaje", ""),
        )


def reconstruir_ruta(padres: dict[Posicion, Posicion | None], meta: Posicion) -> list[Posicion]:
```

```
    """Sigue los punteros a padre desde la meta hasta el inicio y voltea la lista."""
    ruta = []
    actual: Posicion | None = meta
    while actual is not None:
        ruta.append(actual)
        actual = padres[actual]
    ruta.reverse()
    return ruta


def resultado_exitoso(
    nombre: str,
    mapa: Cuadricula,
    padres: dict[Posicion, Posicion | None],
    meta: Posicion,
    expandidos: int,
    generados: int,
    max_frontera: int,
    orden: list[Posicion],
) -> Resultado:
    ruta = reconstruir_ruta(padres, meta)
    return Resultado(
        algoritmo=nombre,
        exito=True,
        ruta=ruta,
        costo=mapa.costo_ruta(ruta),
        nodos_expandidos=expandidos,
        nodos_generados=generados,
        max_frontera=max_frontera,
        orden_expansion=orden,
        mensaje=f"Llegó a la salida {meta}.",
    )


def validar_inicio(mapa: Cuadricula, inicio: Posicion) -> str | None:
    """Devuelve un mensaje de error si el inicio no sirve, o None si está bien."""
    if not mapa.dentro(inicio):
        return "El inicio está fuera del mapa."
    if not mapa.transitable(inicio):
        return "El inicio está sobre un muro o fuego."
    if not mapa.salidas:
        return "El mapa no tiene salidas."
    return None
```

C.2. simulador/algoritmos/no_informados.py (BFS, DFS, UCS)

Archivo: simulador/algoritmos/no_informados.py (líneas 1–149)

```
"""
Búsquedas NO informadas: no usan ninguna pista sobre dónde está la salida.

    BFS  - Búsqueda en amplitud     (cola FIFO)
    DFS  - Búsqueda en profundidad  (pila LIFO)
    UCS  - Búsqueda de costo uniforme (cola de prioridad por costo acumulado g)

Las tres buscan la salida MÁS CERCANA de entre todas las del mapa: cualquier
celda 'S' cumple la prueba de objetivo.
"""

from __future__ import annotations

import heapq
from collections import deque
from itertools import count

from ..cuadricula import Cuadricula, Posicion
from .base import Resultado, resultado_exitoso, validar_inicio


def bfs(mapa: Cuadricula, inicio: Posicion) -> Resultado:
    """
    Búsqueda en amplitud.

    Explora por "capas": primero todo lo que está a 1 paso, luego a 2, etc.
    Encuentra la ruta con MENOS PASOS, pero ignora el costo del humo, así que
    su ruta puede ser más cara que la de UCS o A*.
```

```
29
30      Completa: sí.  Óptima en costo: solo si todos los pasos cuestan igual.
31      """
32      nombre = "BFS"
33      if error := validar_inicio(mapa, inicio):
34          return Resultado(nombre, False, mensaje=error)
35
36      frontera = deque([inicio])
37      padres: dict[Posicion, Posicion | None] = {inicio: None}  # también sirve de "visitados"
38      expandidos = generados = 0
39      max_frontera = 1
40      orden: list[Posicion] = []
41
42      while frontera:
43          actual = frontera.popleft()
44          expandidos += 1
45          orden.append(actual)
46
47          if mapa.es_salida(actual):
48              return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)
49
50          for vecino in mapa.vecinos(actual):
51              if vecino not in padres:
52                  padres[vecino] = actual
53                  frontera.append(vecino)
54                  generados += 1
55          max_frontera = max(max_frontera, len(frontera))
56
57      return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
58                       max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")
59
60
61  def dfs(mapa: Cuadricula, inicio: Posicion) -> Resultado:
62      """
63      Búsqueda en profundidad.
64
65      Sigue un camino hasta el fondo antes de intentar otro. Usa poca memoria,
66      pero la primera salida que encuentra suele dar una ruta larga y cara.
67      Se marca "visitado" al expandir, como en la versión clásica con pila.
68
69      Completa: sí (en grafos finitos con control de visitados).  Óptima: no.
70      """
71      nombre = "DFS"
72      if error := validar_inicio(mapa, inicio):
73          return Resultado(nombre, False, mensaje=error)
74
75      pila: list[Posicion] = [inicio]
76      padres: dict[Posicion, Posicion | None] = {inicio: None}
77      visitados: set[Posicion] = set()
78      expandidos = generados = 0
79      max_frontera = 1
80      orden: list[Posicion] = []
81
82      while pila:
83          actual = pila.pop()
84          if actual in visitados:
85              continue
86          visitados.add(actual)
87          expandidos += 1
88          orden.append(actual)
89
90          if mapa.es_salida(actual):
91              return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)
92
93          # Se apilan al revés para que el PRIMER vecino (arriba) sea el primero en salir.
94          for vecino in reversed(mapa.vecinos(actual)):
95              if vecino not in visitados:
96                  padres[vecino] = actual
97                  pila.append(vecino)
98                  generados += 1
99          max_frontera = max(max_frontera, len(pila))
100
101     return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
102                      max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")
103
```

```
104
105 def ucs(mapa: Cuadricula, inicio: Posicion) -> Resultado:
106     """
107     Búsqueda de costo uniforme (equivale a Dijkstra con parada temprana).
108
109     Siempre expande la celda con menor costo acumulado g(n). Por eso, la
110     primera salida que expande es la de MENOR COSTO total: evita el humo
111     cuando rodear sale más barato.
112
113     Completa: sí.  Óptima en costo: sí (todos los costos son positivos).
114     """
115     nombre = "UCS"
116     if error := validar_inicio(mapa, inicio):
117         return Resultado(nombre, False, mensaje=error)
118
119     desempate = count()
120     frontera: list[tuple[int, int, Posicion]] = [(0, next(desempate), inicio)]
121     g: dict[Posicion, int] = {inicio: 0}
122     padres: dict[Posicion, Posicion | None] = {inicio: None}
123     cerrados: set[Posicion] = set()
124     expandidos = generados = 0
125     max_frontera = 1
126     orden: list[Posicion] = []
127
128     while frontera:
129         costo_actual, _, actual = heapq.heappop(frontera)
130         if actual in cerrados:  # entrada vieja: ya la expandimos con un costo menor
131             continue
132         cerrados.add(actual)
133         expandidos += 1
134         orden.append(actual)
135
136         if mapa.es_salida(actual):
137             return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)
138
139         for vecino in mapa.vecinos(actual):
140             nuevo_g = costo_actual + mapa.costo(vecino)
141             if vecino not in cerrados and nuevo_g < g.get(vecino, float("inf")):
142                 g[vecino] = nuevo_g
143                 padres[vecino] = actual
144                 heapq.heappush(frontera, (nuevo_g, next(desempate), vecino))
145                 generados += 1
146         max_frontera = max(max_frontera, len(frontera))
147
148     return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
149                      max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")
```

C.3. simulador/algoritmos/informados.py (Greedy, A, Hill Climbing)*

Archivo: simulador/algoritmos/informados.py (líneas 1–164)

```
  1 """
  2 Búsquedas INFORMADAS: usan la heurística h(n) = distancia Manhattan a la
  3 salida más cercana para decidir qué explorar primero.
  4
  5     Greedy Best-First - solo mira h(n)              (rápida, no óptima)
  6     A*                - mira f(n) = g(n) + h(n)     (óptima con h admisible)
  7     Hill Climbing     - se mueve al vecino con menor h, sin vuelta atrás
  8                         (puede quedarse atorada en un mínimo local)
  9 """
 10
 11 from __future__ import annotations
 12
 13 import heapq
 14 from itertools import count
 15
 16 from ..cuadricula import Cuadricula, Posicion
 17 from .base import Resultado, resultado_exitoso, validar_inicio
 18
 19
 20 def greedy(mapa: Cuadricula, inicio: Posicion) -> Resultado:
 21     """
 22     Búsqueda voraz primero el mejor (Greedy Best-First).
 23
 24     Expande la celda que "parece" más cerca de una salida según h(n), sin
```

```
    importar cuánto costó llegar ahí. Suele expandir muy pocos nodos, pero
    puede meterse en un callejón o atravesar humo y dar una ruta cara.

    Completa: sí (búsqueda en grafo finita).  Óptima: no.
    """
    nombre = "Greedy"
    if error := validar_inicio(mapa, inicio):
        return Resultado(nombre, False, mensaje=error)

    desempate = count()
    frontera = [(mapa.heuristica(inicio), next(desempate), inicio)]
    padres: dict[Posicion, Posicion | None] = {inicio: None}  # también marca "ya generado"
    cerrados: set[Posicion] = set()
    expandidos = generados = 0
    max_frontera = 1
    orden: list[Posicion] = []

    while frontera:
        _, _, actual = heapq.heappop(frontera)
        if actual in cerrados:
            continue
        cerrados.add(actual)
        expandidos += 1
        orden.append(actual)

        if mapa.es_salida(actual):
            return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)

        for vecino in mapa.vecinos(actual):
            if vecino not in padres:
                padres[vecino] = actual
                heapq.heappush(frontera, (mapa.heuristica(vecino), next(desempate), vecino))
                generados += 1
        max_frontera = max(max_frontera, len(frontera))

    return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
                     max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")


def a_estrella(mapa: Cuadricula, inicio: Posicion) -> Resultado:
    """
    A* (algoritmo principal del proyecto).

    Ordena la frontera por f(n) = g(n) + h(n):
      g(n) = costo real desde el inicio (cuenta el humo como 5),
      h(n) = Manhattan a la salida más cercana (estimación optimista).

    Como h nunca sobreestima (admisible) y es consistente, la primera salida
    que se expande es la de menor costo, igual que UCS, pero explorando
    menos celdas porque h "empuja" la búsqueda hacia las salidas.
    En empate de f se prefiere la celda con menor h (la más cercana a salir).

    Completa: sí.  Óptima en costo: sí.
    """
    nombre = "A*"
    if error := validar_inicio(mapa, inicio):
        return Resultado(nombre, False, mensaje=error)

    desempate = count()
    h0 = mapa.heuristica(inicio)
    frontera = [(h0, h0, next(desempate), inicio)]  # (f, h, orden de llegada, celda)
    g: dict[Posicion, int] = {inicio: 0}
    padres: dict[Posicion, Posicion | None] = {inicio: None}
    cerrados: set[Posicion] = set()
    expandidos = generados = 0
    max_frontera = 1
    orden: list[Posicion] = []

    while frontera:
        _, _, _, actual = heapq.heappop(frontera)
        if actual in cerrados:
            continue
        cerrados.add(actual)
        expandidos += 1
        orden.append(actual)

```

```
        if mapa.es_salida(actual):
            return resultado_exitoso(nombre, mapa, padres, actual, expandidos, generados, max_frontera,
orden)

        for vecino in mapa.vecinos(actual):
            nuevo_g = g[actual] + mapa.costo(vecino)
            if vecino not in cerrados and nuevo_g < g.get(vecino, float("inf")):
                g[vecino] = nuevo_g
                padres[vecino] = actual
                h = mapa.heuristica(vecino)
                heapq.heappush(frontera, (nuevo_g + h, h, next(desempate), vecino))
                generados += 1
        max_frontera = max(max_frontera, len(frontera))

    return Resultado(nombre, False, nodos_expandidos=expandidos, nodos_generados=generados,
                     max_frontera=max_frontera, orden_expansion=orden, mensaje="No hay salida alcanzable.")


def hill_climbing(mapa: Cuadricula, inicio: Posicion) -> Resultado:
    """
    Ascenso de colinas, versión de máxima pendiente (steepest ascent).

    Como queremos MINIMIZAR la distancia, "subir" significa bajar h(n):
    en cada paso mira a todos sus vecinos y se mueve al que tenga menor h,
    pero SOLO si es estrictamente mejor que la celda actual. Si ningún vecino
    mejora, se detiene: está en un mínimo local (por ejemplo, pegado a un muro
    detrás del cual está la salida) y reporta que NO llegó.

    No guarda frontera ni visitados: memoria casi nula, pero sin vuelta atrás.

    Completa: no.  Óptima: no.
    """
    nombre = "Hill Climbing"
    if error := validar_inicio(mapa, inicio):
        return Resultado(nombre, False, mensaje=error)

    actual = inicio
    ruta = [actual]
    expandidos = generados = 0
    orden: list[Posicion] = []

    while True:
        expandidos += 1
        orden.append(actual)
        if mapa.es_salida(actual):
            return Resultado(nombre, True, ruta=ruta, costo=mapa.costo_ruta(ruta),
                             nodos_expandidos=expandidos, nodos_generados=generados,
                             max_frontera=1, orden_expansion=orden,
                             mensaje=f"Llegó a la salida {actual}.")

        vecinos = mapa.vecinos(actual)
        generados += len(vecinos)
        if not vecinos:
            break
        # min() se queda con el primero en caso de empate (orden arriba, abajo, izq, der).
        mejor = min(vecinos, key=mapa.heuristica)
        if mapa.heuristica(mejor) >= mapa.heuristica(actual):
            break  # ningún vecino mejora: mínimo local o meseta
        actual = mejor
        ruta.append(actual)

    return Resultado(nombre, False, ruta=ruta, costo=mapa.costo_ruta(ruta),
                     nodos_expandidos=expandidos, nodos_generados=generados,
                     max_frontera=1, orden_expansion=orden,
                     mensaje=f"Se atoró en un mínimo local en {actual} (h = {mapa.heuristica(actual)}).")
```